City University of Hong Kong 2025-2026 Semester A CS3343 Software Engineering Practice

Design Analysis Report
Group 19
Classic UNO Card Game

Conducted By:

Woo King Hei 58533231

Wong Ka Chung 58533673

Chan Pak Yuen 58535918

Chan Wing Ki 58111083

Ho Sung Hin 58010837

Wong Hei Leong 58537010

1. Introduction

UNO is a well known card game that involves playing colored cards with numbers. The rules are simple, if you play all the cards in your hand, you win. Due to its interactive gameplay and easy-to-understand game rules, it has been immensely popular ever since its release. However, some players, especially the more introverted players, may find trouble finding peers and a suitable location to play UNO. Therefore, we developed a virtual version of UNO, where players can play UNO with CPU players anywhere with their digital device.

2. Use Case Diagram

2.1 UNO System Use Case (Overview)

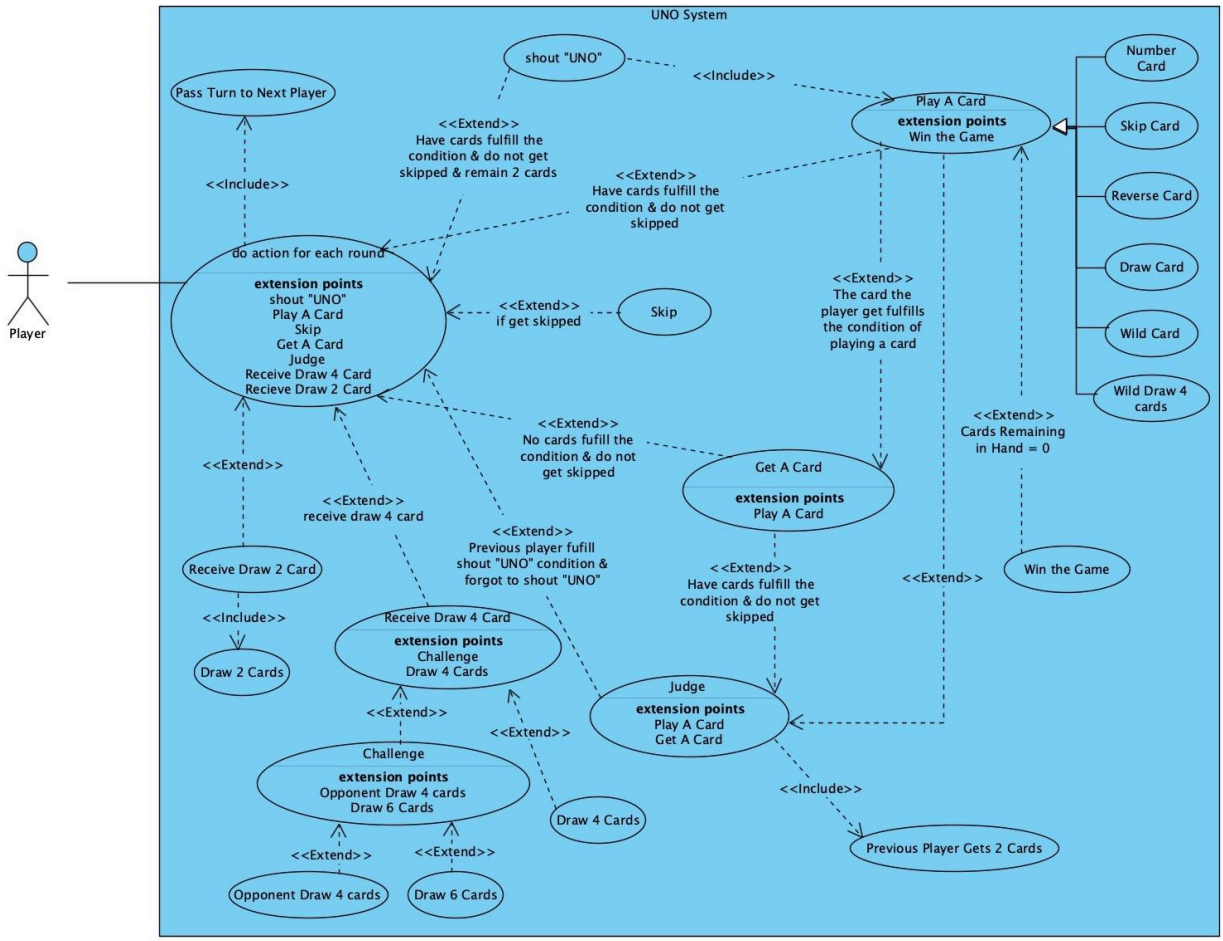


Figure 2. UNO System Use Case

In a game of UNO, the player does action for each round, then passes the turn to the next player. There are various types of actions that players can do depending on the situation. Normally, the player can play a card or get a card from the deck. But if the circumstances match the condition, such as only having two cards left in hand or receiving a draw card from the last player, they are also able to perform other actions such as calling “UNO”.

2.1.1 Shout UNO Use Case

Use Case Name:	Shout UNO	
Actor(s):	Player	
Description:	This use case describes the player action call “UNO” if only two cards are left in hand	
Typical course of events:	**Actor Action**	**System Response**
	Step 1: The player presses the shout UNO button Step 3: The player plays the card Step 4: The player passes turn to next player	Step 2: The system invokes the use case “Shout UNO” Step 5: The system invokes the use case “Pass turn to next player”
Alternative course of events:	N/A	
Precondition:	The player has two cards left in hand, does not get skipped and has a valid card to play	
Postcondition:	The player played a card and now only has one card left in hand	

2.1.2 Play a Card Use Case

Use Case Name:	Play a Card	
Actor(s):	Player	
Description:	This use case describes the player action “Play a Card”	
Typical course of events:	**Actor Action**	**System Response**
	Step 1: The player selects a card Step 2: The player plays the card Step 4: The player passes turn to next player	Step 3: The system invokes the use case “Play a Card” Step 5: The system invokes the use case “Pass turn to next player”
Alternative course of events:	Step 3: [Extension point: If the player has no cards left in hand, invoke use case “Win the Game” and the game ends].	
Precondition:	The player does not get skipped and has a valid card to play	
Postcondition:	The player played a card	

2.1.3 Get a Card Use Case

Use Case Name:	Get a Card	
Actor(s):	Player	
Description:	This use case describes the player action “Get a Card”	
Typical course of events:	**Actor Action**	**System Response**
	Step 1: The player presses get card from deck button	
		Step 2: The system invokes the use case “Get a Card”
	Step 3: The player adds a card from the deck to the hand	
	Step 4: The player passes turn to next player	
		Step 5: The system invokes the use case “Pass turn to next player”
Alternative course of events:	Step 3: [Extension point: If the player gets a card that is valid to play, they can play the card, invoking the use case “Play a Card”].	
Precondition:	The player does not get skipped and has no valid card to play	
Postcondition:	The player gets a card from the deck to the hand	

2.1.4 Skip Use Case

Use Case Name:	Skip	
Actor(s):	Player	
Description:	This use case describes the player action “Skip”	
Typical course of events:	**Actor Action**	**System Response**
	Step 1: The player has received a Skip card from previous player Step 3: The player passes turn to next player	Step 2: The system invokes the use case “Skip” Step 4: The system invokes the use case “Pass turn to next player”
Alternative course of events:	N/A	
Precondition:	The player gets skipped	
Postcondition:	The player automatically passes turn to next player	

2.1.5 Judge Use Case

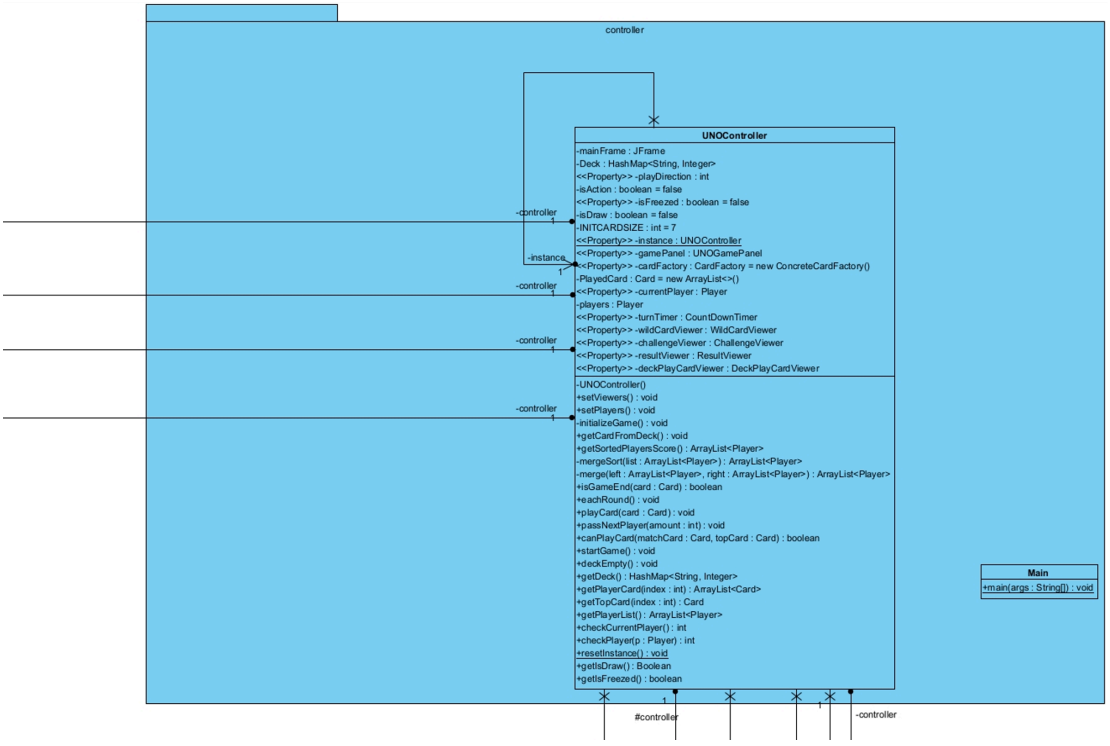
Use Case Name:	Judge	
Actor(s):	Player, Previous Player	
Description:	This use case describes the player action “Judge”, which is the action players can do if the previous player forgot to call “UNO”	
Typical course of events:	**Actor Action**	**System Response**
	Step 1: Player presses the button judge Step 3: The previous player draws two cards	Step 2: The system invokes the use case “Judge” Step 4: Player continues his/her turn, either play a card or draw a card
Alternative course of events:	N/A	
Precondition:	The previous player forgot to call UNO	
Postcondition:	The previous player draws two cards	

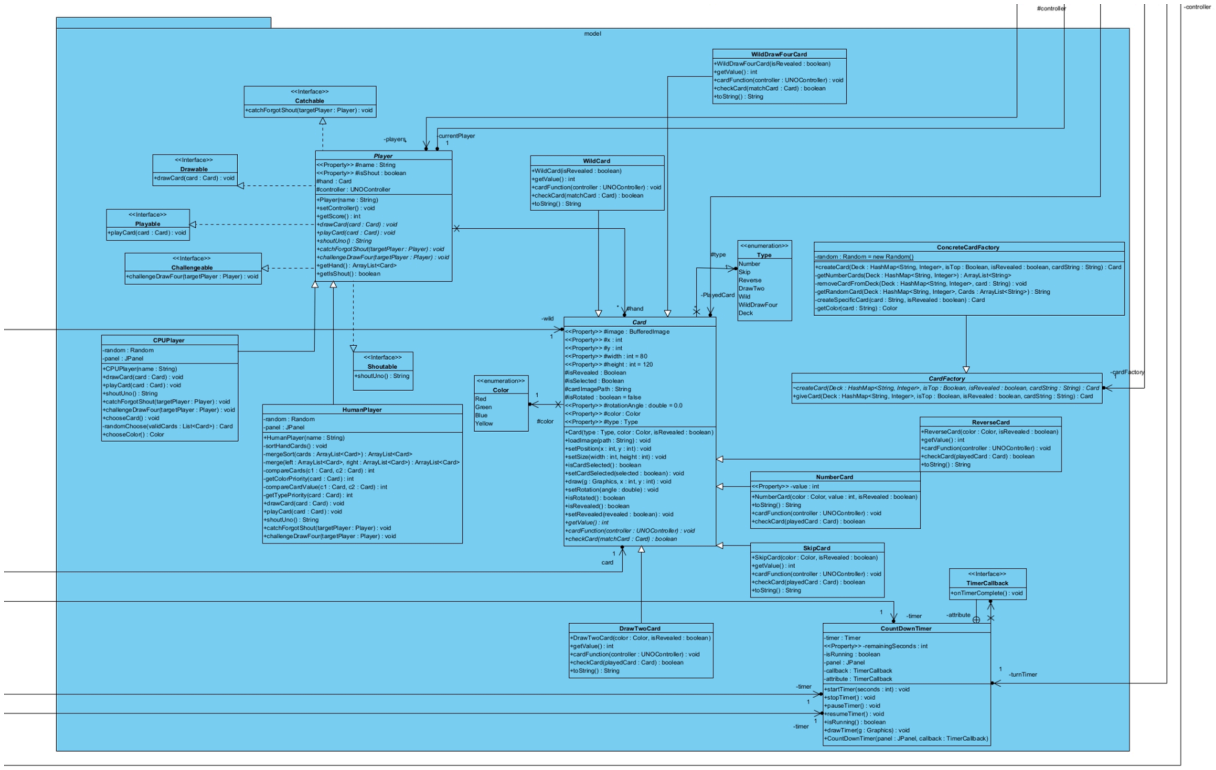
2.1.6 Receive Draw 4 Card Use Case

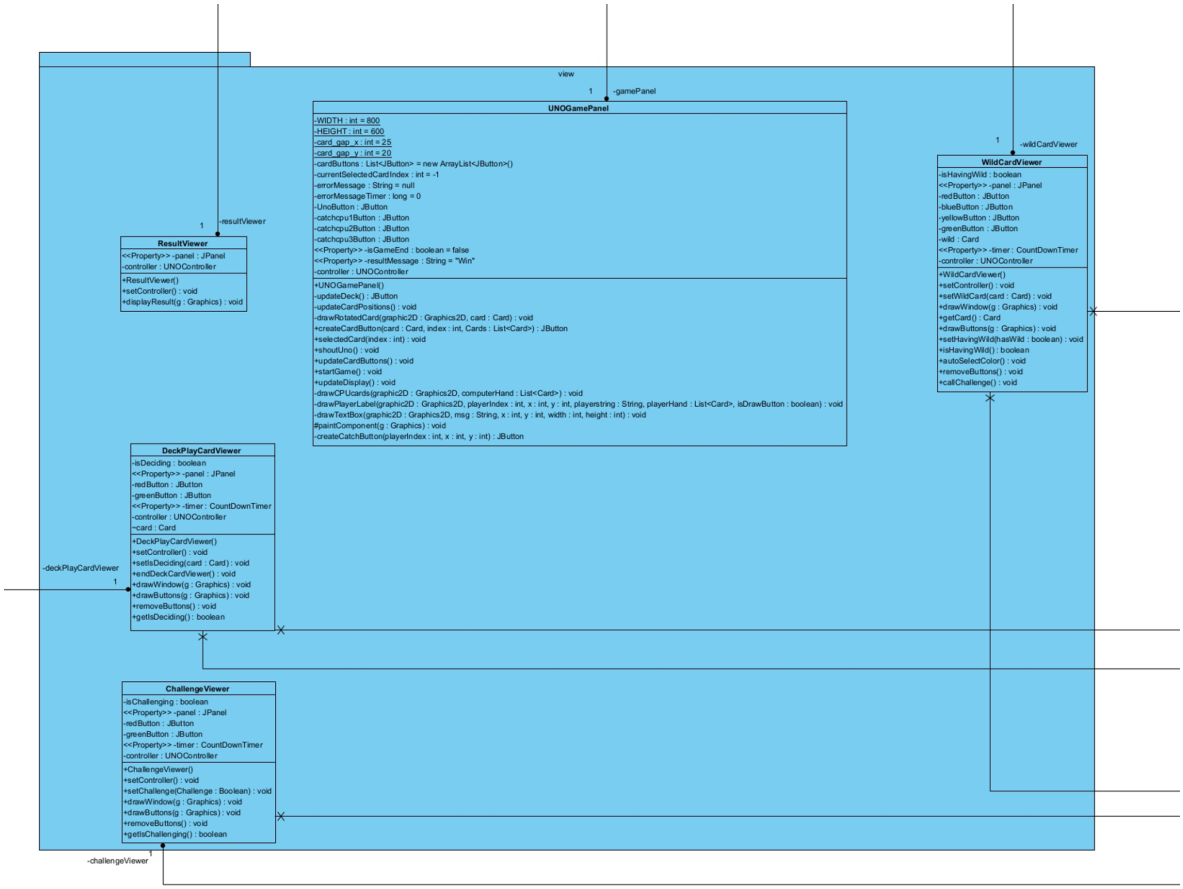
Use Case Name:	Receive Draw 4 Card	
Actor(s):	Player, Previous Player	
Description:	This use case describes the player action “Receive Draw 4 Card”, which is the action players can do if the previous player forgot to call “UNO”	
Typical course of events:	**Actor Action**	**System Response**
	Step 1: Previous Player plays Draw 4 and pass turn Step 3: Player can choose to Draw 4 Cards or Challenge	Step 2: The system invokes the use case "Receive Draw 4”
Alternative course of events:	Step 3: [Extension point: If the player chooses to Draw 4 cards, invoke use case “Draw 4 cards”]. Step 3: [Extension point: If the player challenges, invoke use case “Previous Player Draw 2 cards” or “Draw 6 cards” depending on challenge result].	
Precondition:	The previous player plays Draw 4 Card	
Postcondition:	Player receiving the Draw 4 card either challenge or draw cards	

3. Class Diagram

3.1 Overview





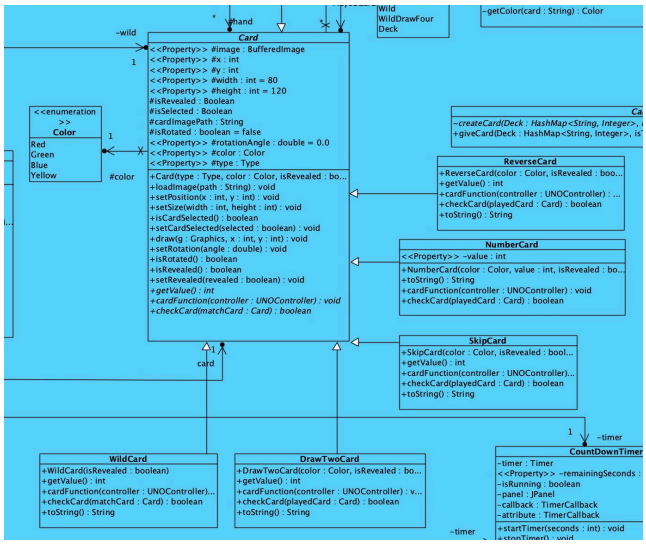

view
1
-gamePanel
UNOGamePanel
-WIDTH : int = 800
-HEIGHT : int = 600
-card_gap_x : int = 25
-card_gap_y : int = 20
-cardButtons : List<JButton> = new ArrayList<JButton>()
-currentSelectedCardIndex : int = -1
-errorMessage : String = null
-errorMessageTimer : long = 0
-UnoButton : JButton
-catchcpu1Button : JButton
-catchcpu2Button : JButton
-catchcpu3Button : JButton
<<Property>> -isGameEnd : boolean = false
<<Property>> -resultMessage : String = "Win"
-controller : UNOController
+UNOGamePanel()
-updateDeck() : JButton
-updateCardPositions() : void
-drawRotatedCard(graphic2D : Graphics2D, card : Card) : void
+createCardButton(card : Card, index : int, Cards : List<Card>) : JButton
+selectedCard(index : int) : void
+shoutUno() : void
+updateCardButtons() : void
+startGame() : void
+updateDisplay() : void
-drawCPUcards(graphic2D : Graphics2D, computerHand : List<Card>) : void
-drawPlayerLabel(graphic2D : Graphics2D, playerIndex : int, x : int, y : int, playerstring : String, playerHand : List<Card>, isDrawButton : boolean) : void
-drawTextBox(graphic2D : Graphics2D, msg : String, x : int, y : int, width : int, height : int) : void
#paintComponent(g : Graphics) : void
-createCatchButton(playerIndex : int, x : int, y : int) : JButton
1
-resultViewer
ResultViewer
<<Property>> -panel : JPanel
-controller : UNOController
+ResultViewer()
+setController() : void
+displayResult(g : Graphics) : void
1
-wildCardViewer
WildCardViewer
-isHavingWild : boolean
<<Property>> -panel : JPanel
-redButton : JButton
-blueButton : JButton
-yellowButton : JButton
-greenButton : JButton
-wild : Card
<<Property>> -timer : CountDownTimer
-controller : UNOController
+WildCardViewer()
+setController() : void
+setWildCard(card : Card) : void
+drawWindow(g : Graphics) : void
+getCard() : Card
+drawButtons(g : Graphics) : void
+setHavingWild(hasWild : boolean) : void
+isHavingWild() : boolean
+autoSelectColor() : void
+removeButtons() : void
+callChallenge() : void
DeckPlayCardViewer
-isDeciding : boolean
<<Property>> -panel : JPanel
-redButton : JButton
-greenButton : JButton
<<Property>> -timer : CountDownTimer
-controller : UNOController
~card : Card
+DeckPlayCardViewer()
+setController() : void
+setIsDeciding(card : Card) : void
+endDeckCardViewer() : void
+drawWindow(g : Graphics) : void
+drawButtons(g : Graphics) : void
+removeButtons() : void
+getIsDeciding() : boolean
-deckPlayCardViewer
1
ChallengeViewer
-isChallenging : boolean
<<Property>> -panel : JPanel
-redButton : JButton
-greenButton : JButton
<<Property>> -timer : CountDownTimer
-controller : UNOController
+ChallengeViewer()
+setController() : void
+setChallenge(Challenge : Boolean) : void
+drawWindow(g : Graphics) : void
+drawButtons(g : Graphics) : void
+removeButtons() : void
+getIsChallenging() : boolean
1
-challengeViewer

4. Design Principles

In the class diagram above, we implemented a number of design principles in the code design. In the following section we will go in depth on the principles we followed in our code base .We aim to make the code more readable and organised in order to facilitate collaboration and future modifications, such as adding new features and game mechanics.

4.1 Open Closed Principle (OCP)

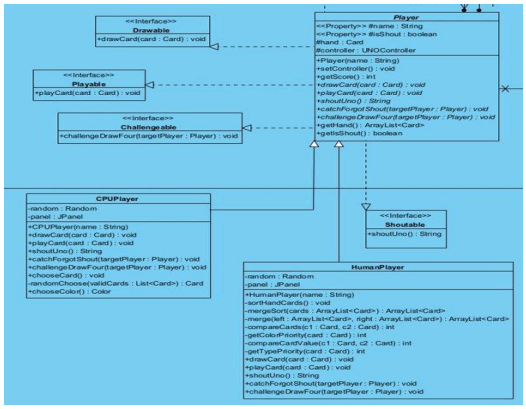
Open Closed Principle states that the code should be “Open for Extension, Closed for Modification”. New functions should be implemented through adding new code instead of changing the existing code.



In Card.java, this principle is implemented. The code implements an abstract class Card and creates cards by concrete class like Draw2Card and NumberCard so that we can add new card types by just adding a new Card Type, instead of changing the existing Card.java code.

4.2 Liskov Substitution Principle (LSP)

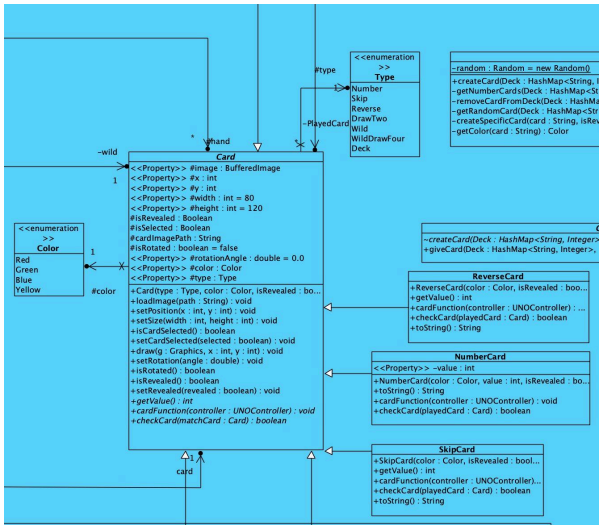
Liskov Substitution Principle (LSP) states that the parent class should contain functions or variables that are applicable to all child classes. In other words, child class objects should be completely replaceable of their parent class object without breaking the application.



In the code, the Player class object is completely replaceable with HumanPlayer and CPUPlayer class objects. Also, these two classes contain unique functions that only apply to their own class object. For example, CPUPlayer has the function randomChoose() to randomly choose a card to play which is unique to their own class and not applicable in HumanPlayer.

4.3 Dependency Inversion Principle (DIP)

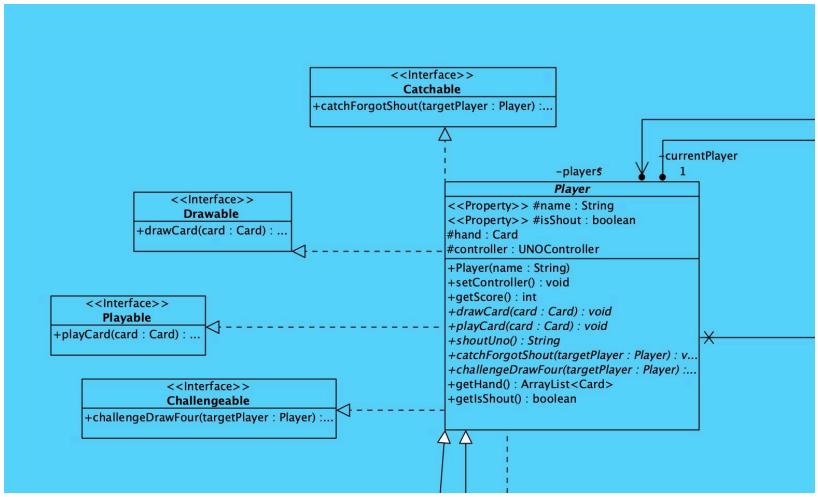
Dependency Inversion Principle states that modules should not depend on low level concrete modules. Instead, they should depend on high level abstractions and interfaces.

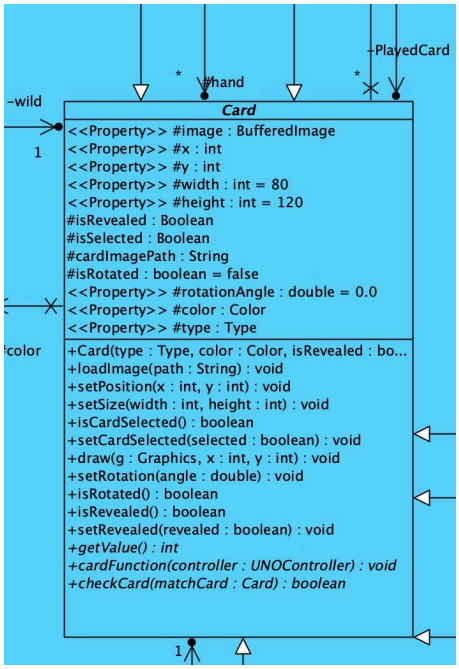


In the code, all concrete Card classes like WildCard and ReverseCard depend on abstract class Card, which handles the general card logic. This makes it more convenient if we need to change the game logic for future updates and or implement new types of cards.

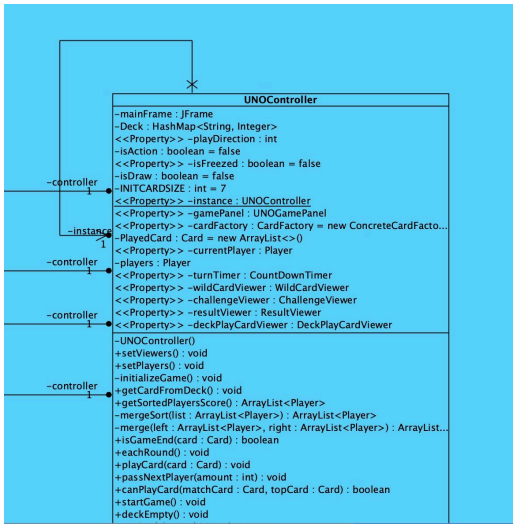
4.4 Single Responsibility Principle (SRP)

Single Responsibility Principle states that each component only has one responsibility.





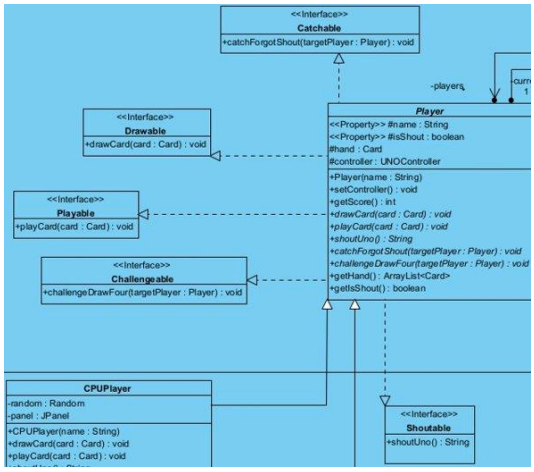
In our code, the core components all only have one high level responsibility. The Player component is responsible for Player operations such as drawing and playing cards whereas the Card component is responsible for card function and card creation. This ensures low coupling, and we only need to modify the relevant components for future changes.



Although UNOController is responsible for a large amount of operations, its main responsibility is still managing the game state and logic. If the game logic or operation needs to be modified in the future, we only need to change code from this component.

4.5 Interface Segregation Principle (ISP)

Interface Segregation Principle is a design principle where multiple specific interfaces should be implemented instead of one large generic interface, and interfaces should only implement methods they actually use.



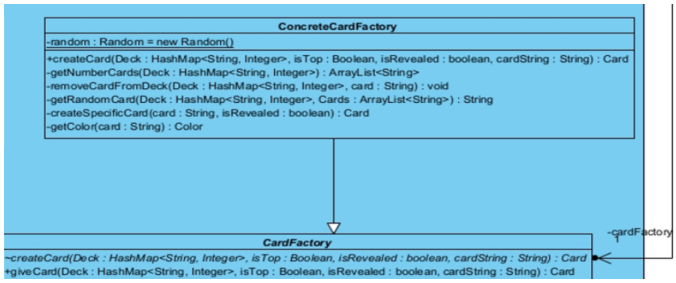
In our player component, 5 different specific interfaces were implemented instead of one Player interface. This facilitates low coupling and increases maintainability as only that specific interface needs to be modified when a game component needs to be updated.

5. Design Patterns

In the code, we also implemented some design patterns which are beneficial for improving efficiency. This section will illustrate some of the design principles implemented in our code base, and explain how they can further enhance readability, collaboration and future extensions.

5.1 Factory Pattern

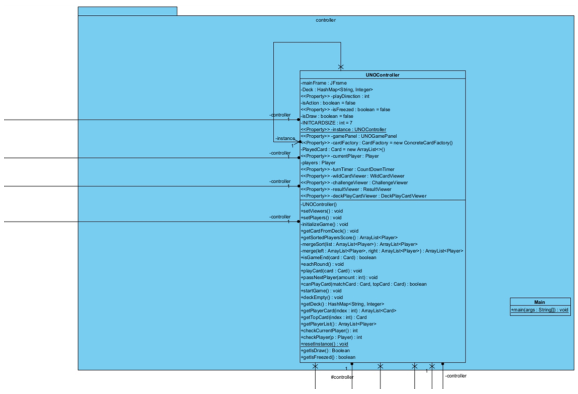
Factory Pattern ensures that concrete objects are created at runtime. Using this pattern, the method for creating new objects is located at the “Factory” class object, which will decide the type of object to create at runtime.



In our code, a Factory class CardFactory was implemented. It contains the function createCard() which will initialize cards with different types at runtime, such as WildCard and WildDrawFourCard. By using this pattern, the card creation logic is encapsulated in ConcreteCardFactory, which improves code organization.

5.2 Singleton Pattern

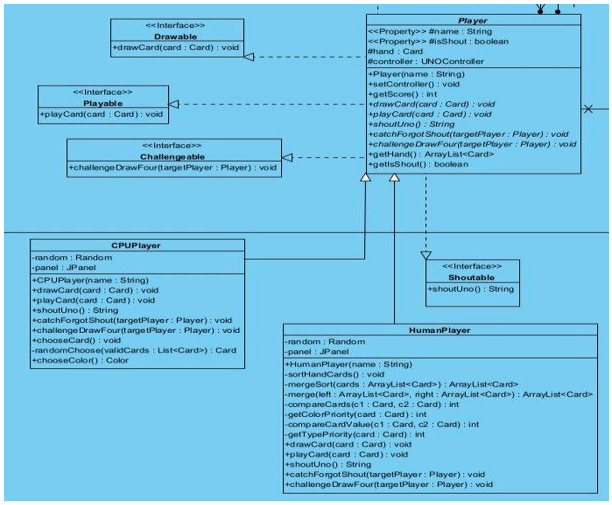
By implementing Singleton Pattern on a class object, it makes sure that there is only one instance of that class object by making the constructor private and one global access point to that object with the function getInstance().



In our code, the UNOController is responsible for the entire game logic. To ensure there is only one single instance of UNOController, a Singleton pattern is implemented. A private constructor is provided as well as an Instance variable and a getInstance() function for global access. This design pattern can prevent multiple instances of UNOController being created, which is not the preferred functionality.

5.3 Strategy Pattern

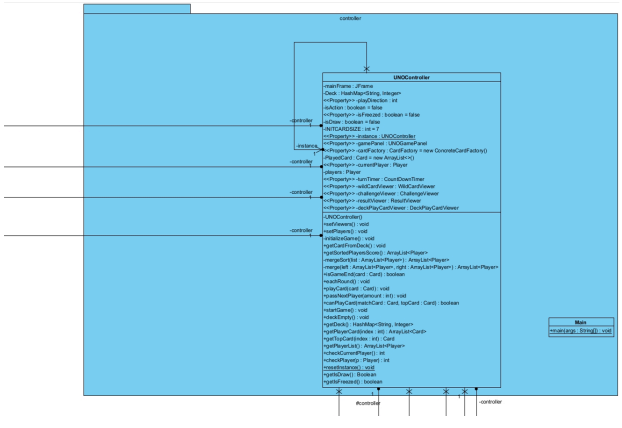
Strategy pattern implements different object behavior in different class objects, and decides which behavior to use during runtime.



In our code, the strategy pattern is implemented at Player.java. It contains different strategies such as drawCard() and playCard(). This way, HumanPlayer and CPUPlayer are able to use different logic for drawCard() and playCard() during runtime, and modifying the logic for HumanPlayer's drawCard()or playCard() will not affect the corresponding function for CPUPlayer, and vice versa.

5.4 Facade Pattern

Facade pattern provides a simple user interface for users to interact with the complex functions inside the code.

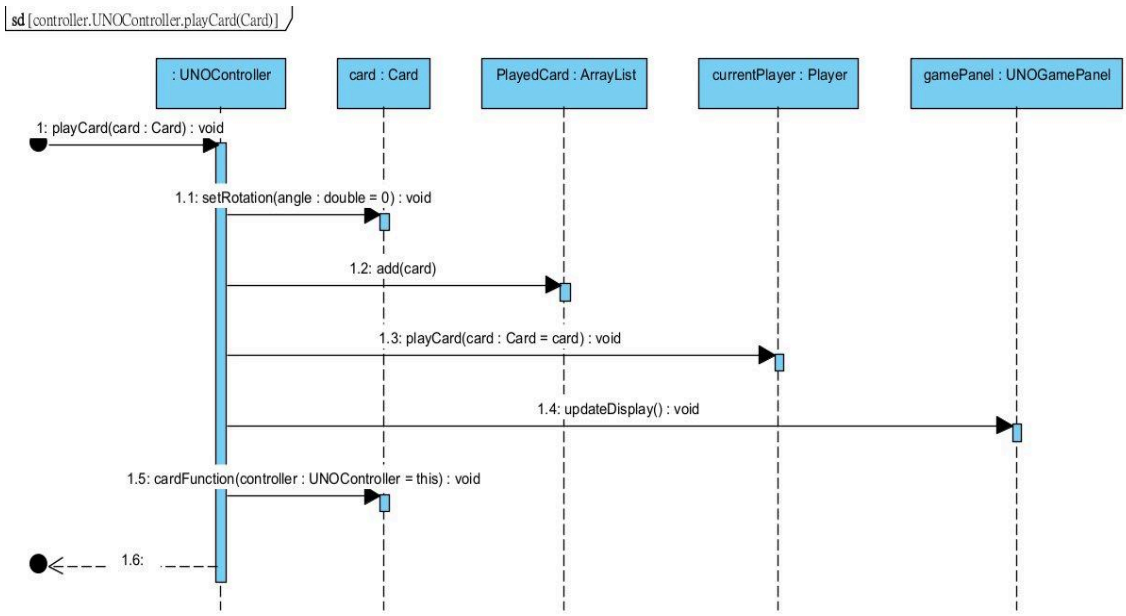


In Main.java , players can start the program by simply calling UNO.getInstance() which means that UNOController gives a single entry point. This prevents players/users from directly

interacting with the complex functions in UNOController, instead they can access them by UNO.getInstance().

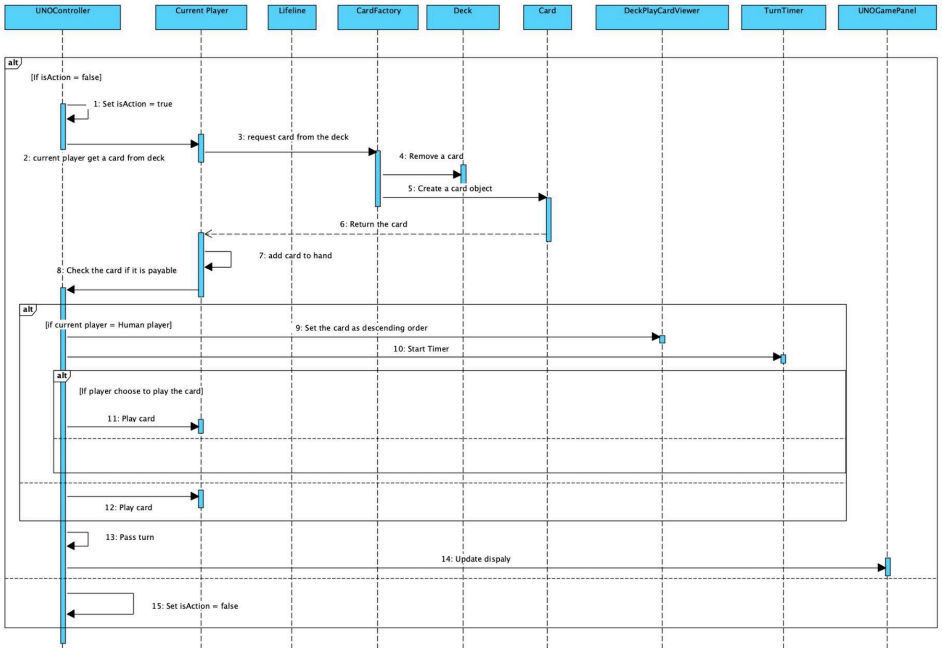
6. Sequence Diagrams

6.1 Playing a Card



When playing a card, UNOController will first rotate the selected card, then add the card to the PlayedCard ArrayList. Once the card gets played, the updateDisplay() function will be called to display the played card in the center of the table. Finally, the cardFunction() of the played card will execute, updating the game state.

6.2 Drawing a Card



When the current player draws a card, UNOController first sets the isAction flag to true, meaning a player is in action. The current Player object will then request a Card from the deck, the deck (CardFactory) will create a Card object at run time and return the card to the player. After adding the card to the Player's hand, UNOController will check if the card is playable, if so, UNOController will call TurnTimer to start a 30 second countdown timer for the player to play the card. After playing the card, UNOController will pass the turn to the next player, and set the isAction flag back to false.